

PROJECT REPORT

STUDYSMART : AN INTELLIGENT STUDENT ASSISTANT

Submitted By

Name: ANANYA SINHA

Registration Number: 25BAI11375

University: VIT BHOPAL UNIVERSITY

Branch: COMPUTER SCIENCE AND
ENGINEERING

Program: CSE-AIML

Course: PROGRAMMING IN JAVA
(CSE-2006)

TABLE OF CONTENTS

- 1.Introduction
- 2.Abstract
- 3.Problem Statement
- 4.Objectives
- 5.Requirements
- 6.Technologies Used
- 7.System Architecture
- 8.Project Workflow
- 9.Project Structure
10. Implementation
11. Graphical User Interface
12. Testing and Validation
13. Sample Execution
14. Challenges Faced
15. Learning Outcomes
16. Future Enhancements
17. Conclusion
18. References

1. INTRODUCTION

As students, we usually have to manage a lot of things at the same time, such as study schedules, assignments, revision, practice questions, and notes. Keeping track of everything separately can sometimes become difficult.

The idea behind **StudySmart** came from this simple problem. I wanted to create a small application where different study-related activities could be managed from one place.

StudySmart is a Java-based desktop application developed using **Java Swing**. It provides features such as creating study tasks, attempting quizzes, checking performance, getting basic study recommendations, and saving notes.

The project also helped me understand how the concepts learned in Programming in Java can be used together to build an actual application instead of writing separate programs for individual concepts.

2. ABSTRACT

StudySmart is a student study management application developed using Java. The application combines some common study activities into one system.

The application has a dashboard from which the user can access the Study Planner, Quiz, Performance, and Notes sections.

The Study Planner allows the user to create tasks by entering the subject, topic, duration, and difficulty. Tasks can also be marked as completed or removed.

The Quiz module contains multiple-choice questions and calculates the score after the quiz is completed. The Performance section displays scores and provides basic analysis such as average performance and weakest subject.

The application also contains a simple rule-based Recommendation Engine. It gives study suggestions according to the score, difficulty of the topic, and available study time.

The Notes section uses Java file handling to save and read notes from a text file.

Overall, this project demonstrates the practical use of Java concepts such as classes and objects, collections, methods, exception handling, file handling, packages, and Java Swing.

3. PROBLEM STATEMENT

Students use different methods to manage their studies. A study timetable may be maintained separately, practice questions may be attempted somewhere else, and notes may be stored in different files or notebooks.

Because of this, it can be difficult to keep these activities organized.

The purpose of StudySmart is to provide a simple application where basic study planning, quiz practice, performance checking, recommendations, and note management are available in one place.

4. OBJECTIVES

The main objectives of this project are:

- To create a useful student-oriented application using Java.
 - To apply Object-Oriented Programming concepts in a practical project.
 - To create and manage study tasks.
 - To provide a simple quiz system for practice.
 - To calculate quiz scores automatically.
 - To display basic performance information.
 - To identify the weakest subject from the available results.
 - To provide simple study recommendations.
 - To save and retrieve notes using file handling.
 - To create an easy-to-use graphical interface using Java Swing.
 - To understand how different Java classes can work together in one project.
-

5. REQUIREMENTS

5.1 Hardware Requirements

The project does not require any special hardware.

The basic requirements are:

- Laptop or desktop computer
- Minimum 4 GB RAM
- Keyboard and mouse
- Sufficient storage space for Java and development tools

5.2 Software Requirements

- Java Development Kit (JDK)
 - Visual Studio Code
 - Java Extension Pack
 - Windows/Linux/macOS
 - GitHub account for uploading the project.
-

6. TECHNOLOGIES USED

Technology	Use in Project
Java	Main programming language
Java Swing	Creating the graphical interface
ArrayList	Storing tasks, questions and results

Java I/O

Saving and reading notes

Exception Handling Handling invalid inputs and file errors

VS Code

Writing and testing the code

JDK

Compiling and running the application

GitHub

Project storage and version control

7. SYSTEM ARCHITECTURE

I divided the project into different parts so that the code would be easier to understand and manage.

The project mainly contains **Model, Service, and GUI** packages.

Model : The Model package contains the classes used to store information.

It contains:

- `Student.java`
- `Subject.java`
- `StudyTask.java`
- `QuizQuestion.java`
- `QuizResult.java`

Service : The Service package contains the main logic of the application.

It contains:

- `StudyPlanner.java`
- `QuizManager.java`

- `PerformanceAnalyzer.java`
- `RecommendationEngine.java`
- `NotesManager.java`

GUI : The GUI package contains the screens displayed to the user.

It contains:

- `DashboardFrame.java`
- `PlannerPanel.java`
- `QuizPanel.java`
- `PerformancePanel.java`
- `NotesPanel.java`

The `Main.java` file is used to start the application.

• System Architecture Diagram

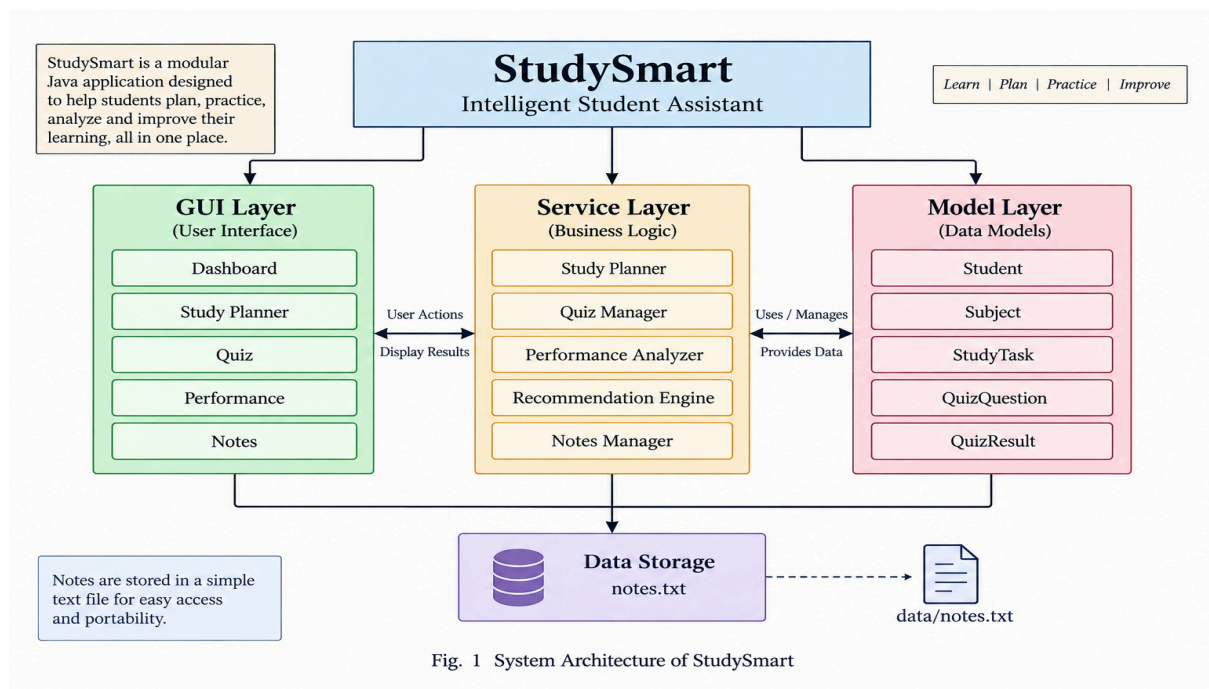


Figure 1: System Architecture of StudySmart

8. PROJECT WORKFLOW

The application follows a simple workflow. First, the user starts the application and the StudySmart dashboard appears.

From the dashboard, the user can choose the required section.

In the Study Planner, the user can enter a subject, topic, duration, and difficulty and create a study task.

In the Quiz section, the user answers multiple-choice questions. After the last question, the application calculates the result.

The Performance section displays available quiz results and calculates the average score. It also identifies the weakest subject.

The Recommendation Engine uses the available score and other inputs to provide a simple study suggestion.

The Notes section allows the user to write notes and save them. These notes are stored in a text file inside the **data** folder.

Application Workflow Diagram

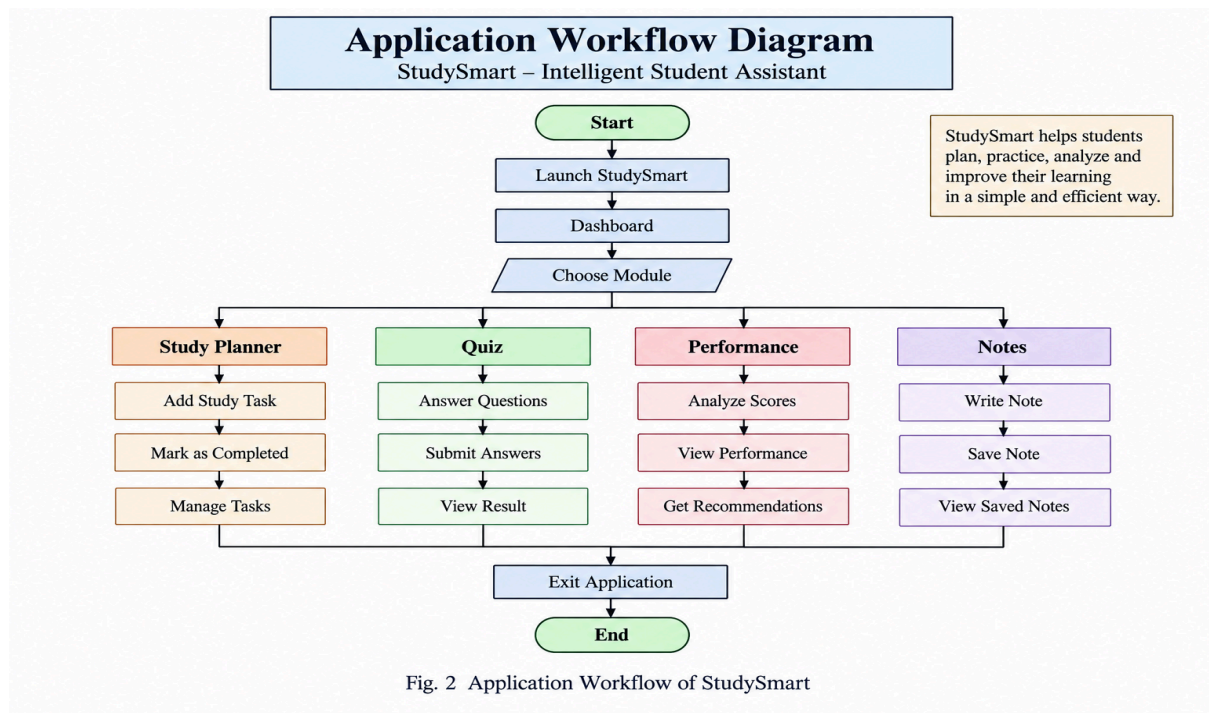


Figure 2: Application Workflow

9. PROJECT STRUCTURE

The project has been organized into different folders and packages.

StudySmart

```
|
|
|— src
|   └─ studysmart
|       └─ Main.java
|           |
|           └─ model
|               └─ Student.java
|               └─ Subject.java
|               └─ StudyTask.java
|               └─ QuizQuestion.java
|               └─ QuizResult.java
|               |
|               └─ service
|                   └─ StudyPlanner.java
|                   └─ QuizManager.java
|                   └─ PerformanceAnalyzer.java
|                   └─ RecommendationEngine.java
|                   └─ NotesManager.java
|                   |
|                   └─ gui
|                       └─ DashboardFrame.java
```

```
|      |— PlannerPanel.java
|      |— QuizPanel.java
|      |— PerformancePanel.java
|      └— NotesPanel.java
|
|— data
|  └— notes.txt
|
|— screenshots
|— documentation
|— README.md
|— statement.md
└— Project_Report.pdf
```

This structure keeps the different parts of the application separate instead of putting all the code into one Java file.

10. UML CLASS DESIGN

The project uses several classes, with each class having a specific purpose.

For example, **StudyTask** represents a study task, while **StudyPlanner** manages multiple study tasks.

Similarly, **QuizQuestion** stores a question and its options, while **QuizManager** manages the quiz questions.

QuizResult stores the result of a quiz and **PerformanceAnalyzer** works with the results to calculate performance information.

UML Class Diagram

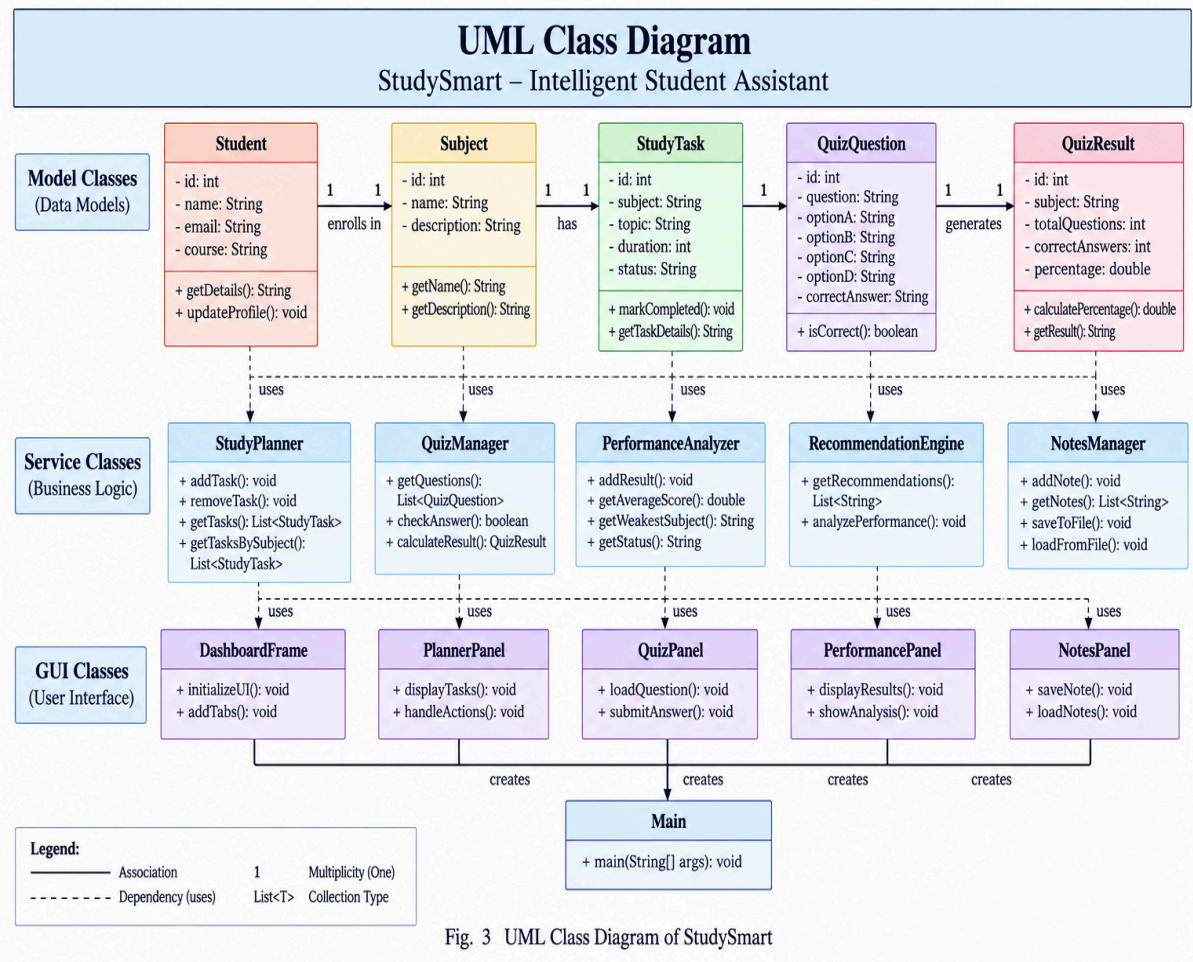


Fig. 3 UML Class Diagram of StudySmart

Figure 3: UML Class Diagram of StudySmart

11. IMPLEMENTATION

❖ 11.1 Student Class

The **Student** class stores basic information about a student. It contains:

- Student ID
- Name
- Email
- Course

Private variables, constructors, getters, and setters are used in the class. This helped in implementing the concept of **encapsulation**.

❖ 11.2 Subject Class

The **Subject** class stores the subject name and score. It also contains a method that checks the score and returns a performance status. The status is divided into:

- Strong
- Moderate
- Needs Improvement

This gives a simple way of representing subject performance.

❖ 11.3 StudyTask Class

The **StudyTask** class represents a single study task. A task contains:

- Subject
- Topic
- Duration
- Difficulty
- Completion status

Whenever a new task is created, its initial status is **Pending**. The status can later be changed to **Completed**.

❖ 11.4 StudyPlanner Class

The Study Planner manages all the study tasks using an **ArrayList**. The class provides methods for:

- Adding tasks
- Removing tasks
- Marking tasks as completed
- Getting all tasks
- Counting total tasks
- Counting completed tasks

Using an **ArrayList** made it easier to store multiple tasks and modify them when required.

❖ 11.5 QuizQuestion Class

The `QuizQuestion` class stores one multiple-choice question. Each question has four options:

- Option A
- Option B
- Option C
- Option D

The correct answer is also stored. The `isCorrect()` method is used to check whether the answer selected by the user is correct.

❖ 11.6 QuizManager Class

The `QuizManager` class is responsible for managing the quiz questions. A list of Java-related questions is loaded when the quiz manager is created.

After the quiz is completed, the manager creates a `QuizResult` object containing the score.

❖ 11.7 QuizResult Class

The `QuizResult` class stores:

- Subject
- Total number of questions
- Number of correct answers

The percentage is calculated using:

$$\text{Percentage} = (\text{Correct Answers} / \text{Total Questions}) \times 100$$

The class also checks for zero total questions to avoid division-related problems.

❖ 11.8 PerformanceAnalyzer Class

The `PerformanceAnalyzer` class is used to work with quiz results. It calculates:

- Average score
- Performance status
- Weakest subject

The results are stored using an `ArrayList`. This part of the project helped in understanding how collections can be used to process multiple objects.

❖ 11.9 RecommendationEngine Class

The Recommendation Engine is a simple rule-based component. It uses:

- Score
- Topic difficulty
- Available study time

Based on these values, it returns a suitable study suggestion.

For example, if the score is low, the application suggests revising basic concepts and practicing more. This is not a machine-learning model. It is implemented using simple Java conditions and rules.

❖ 11.10 NotesManager Class

The `NotesManager` class handles the saving and reading of notes. The notes are stored in:

data/notes.txt

The class uses Java's file handling classes such as:

- `File`
- `FileWriter`
- `BufferedWriter`
- `FileReader`
- `BufferedReader`

Exception handling is also used so that file-related errors can be handled without crashing the application.

12. GRAPHICAL USER INTERFACE

The graphical interface of StudySmart was created using **Java Swing**.

The main window contains five tabs:

1. Dashboard
2. Study Planner
3. Quiz
4. Performance
5. Notes

The purpose of the GUI was to make the different features easier to access instead of requiring the user to interact with the program through the command line.

13. DASHBOARD

The Dashboard is the first screen shown when the application starts. It displays the StudySmart title and provides access to the different modules.

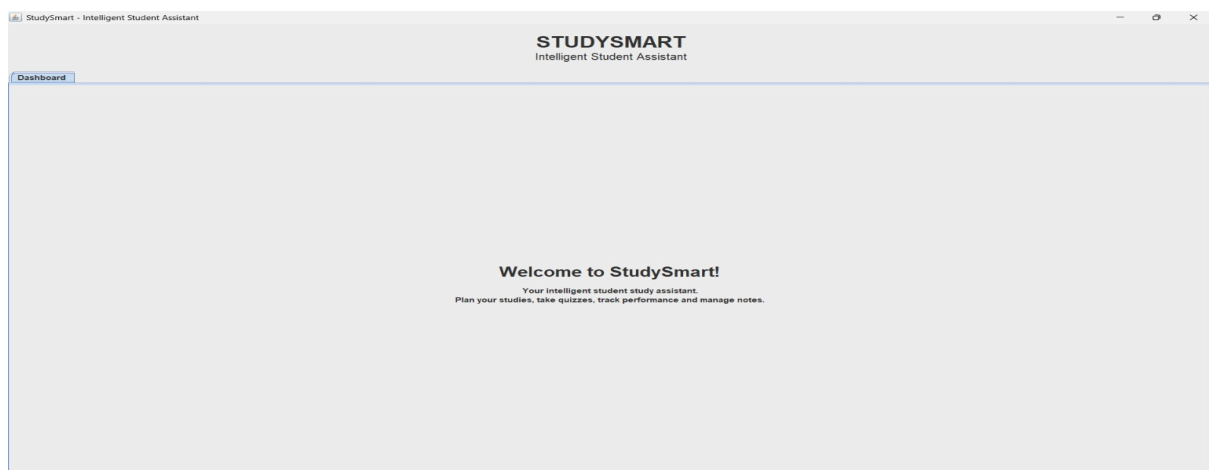


Figure 4: StudySmart Dashboard

14. STUDY PLANNER

The Study Planner allows the user to enter information about a study task. The user enters:

- Subject
- Topic
- Duration
- Difficulty

After clicking **Add Study Task**, the task appears in the table. The user can select a task and mark it as completed or remove it.

Subject	Topic	Duration (min)	Difficulty	Status
JAVA	INHERITANCE	60	Hard	Completed
DSA	ARRAYS	45	Medium	Pending

Figure 5: Study Planner Interface

15. QUIZ MODULE

The Quiz module displays one question at a time. The user selects one of the four options and clicks **Submit Answer**.

After answering all the questions, the application displays the number of correct answers and the final percentage.

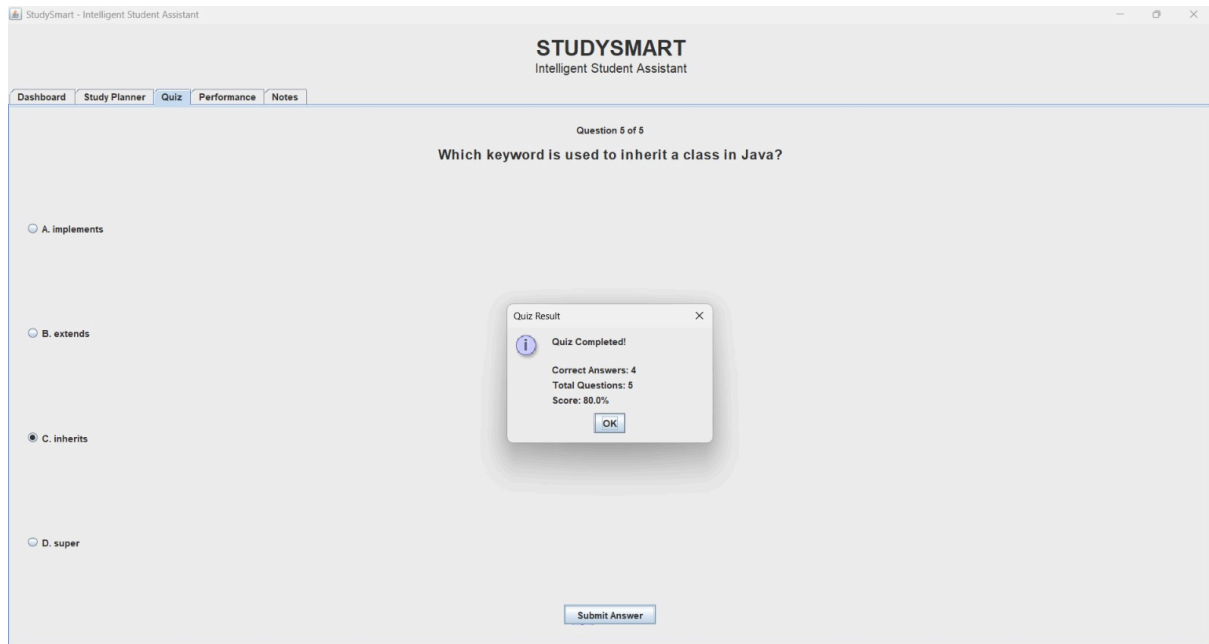


Figure 6: Quiz Interface

16. PERFORMANCE MODULE

The Performance module displays the available subject scores. It also shows:

- Average score
- Performance status
- Weakest subject

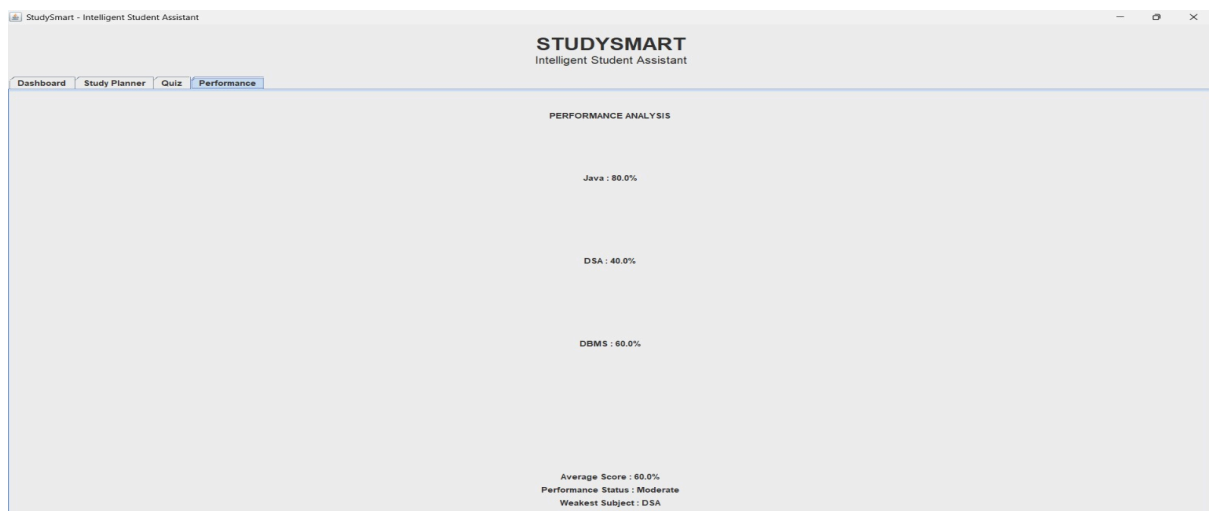
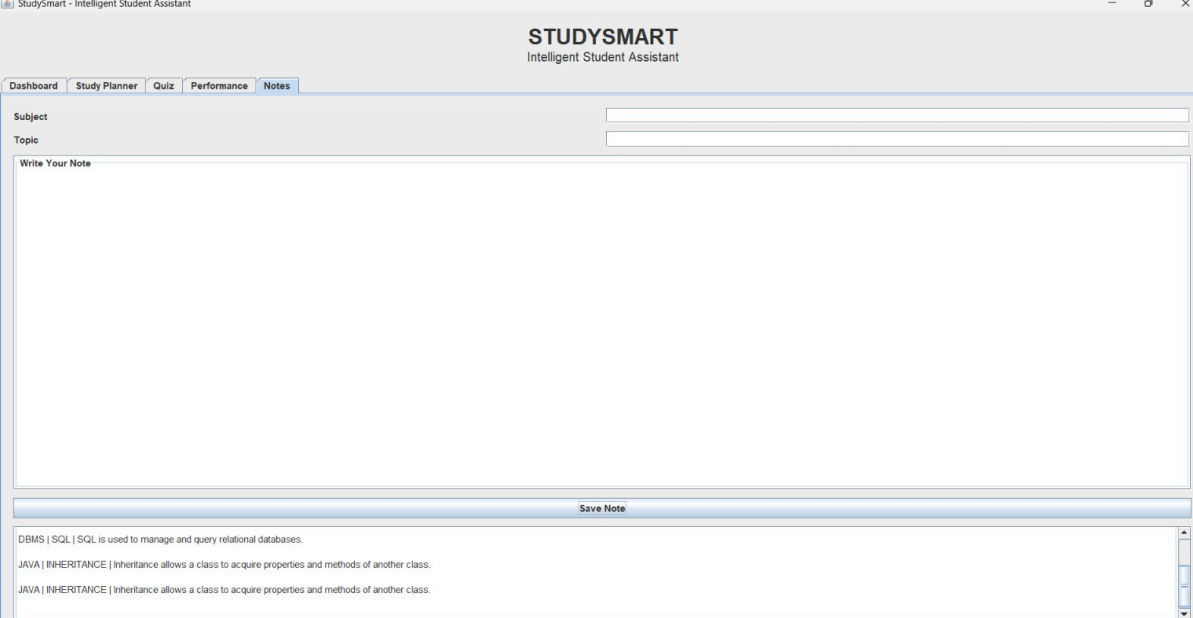


Figure 7: Performance Analysis Interface

17. NOTES MODULE

The Notes module provides a simple way to save study notes. The user enters the subject, topic, and note content and then clicks **Save Note**. The saved notes are displayed in the application and are also stored in the `notes.txt` file.



The screenshot shows the 'STUDYSMART Intelligent Student Assistant' application window. It features a navigation bar with tabs for 'Dashboard', 'Study Planner', 'Quiz', 'Performance', and 'Notes'. The 'Notes' tab is active. Below the navigation bar, there are input fields for 'Subject' and 'Topic'. A large text area labeled 'Write Your Note' is provided for the user to enter their notes. At the bottom of the interface, there is a 'Save Note' button. Below the button, a list of saved notes is displayed, including entries about DBMS | SQL and JAVA | INHERITANCE.

Figure 8: Notes Interface

18. DATA STORAGE

StudySmart uses a simple text file instead of a database for storing notes.

The storage structure is:

data

└─ notes.txt

Whenever a note is saved, it is added to the file.

When the application starts, the existing notes can be read from the file and displayed.

Data Storage Diagram

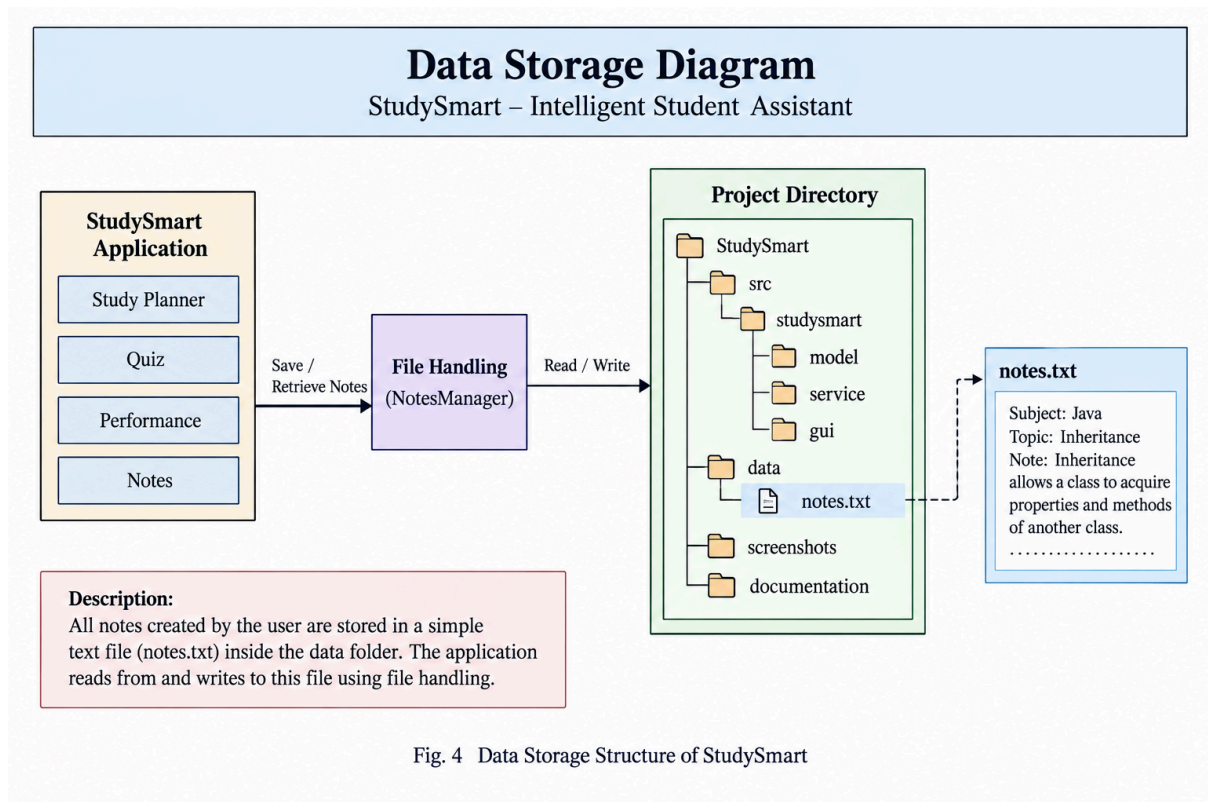


Figure 9: Data Storage Design

19. TESTING AND VALIDATION

Testing was performed after implementing the individual modules as well as after integrating the GUI.

Test ID	Module	Test Performed	Result
T01	Student	Display student information	Passed
T02	Subject	Check performance status	Passed
T03	Study Task	Create and display task	Passed

T04	Study Planner	Add a study task	Passed
T05	Study Planner	Mark task completed	Passed
T06	Study Planner	Remove task	Passed
T07	Quiz	Select and submit answer	Passed
T08	Quiz Result	Calculate quiz percentage	Passed
T09	Performance	Calculate average score	Passed
T10	Performance	Find weakest subject	Passed
T11	Recommendation	Generate recommendation	Passed
T12	Notes	Save a note	Passed
T13	Notes	Read saved notes	Passed
T14	GUI	Open all application tabs	Passed

20. ERROR HANDLING

Some basic validation and error handling has been included in the application.

- **Study Planner** :The application does not allow an empty subject or topic. The duration must be entered as a number and must be greater than zero.If invalid input is entered, an appropriate message is displayed.
 - **Quiz** :The user must select an option before submitting an answer.
 - **Notes** :The user must enter the subject, topic, and note before saving.
 - **File Handling** :File operations are placed inside try-catch blocks so that IOException can be handled properly.
-

21. SAMPLE EXECUTION

Study Planner

Example:

Subject : Java

Topic : Inheritance

Duration : 60 minutes

Difficulty : Hard

The task is displayed as:

Java | Inheritance | 60 minutes | Hard | Pending

After selecting the task and clicking Mark Completed:

Java | Inheritance | 60 minutes | Hard | Completed

Quiz

Sample result:

Quiz Completed!

Correct Answers: 4

Total Questions: 5

Score: 80.0%

Performance

Sample output:

Java : 80.0%

DSA : 40.0%

DBMS : 60.0%

Average Score : 60.0%

Performance Status : Moderate

Weakest Subject : DSA

Notes

Example:

Java | Inheritance |

Inheritance allows one class to acquire properties
and methods of another class.

22. FINAL APPLICATION

After completing the individual modules, all the components were connected to the main StudySmart dashboard. The final application provides access to the Study Planner, Quiz, Performance, and Notes modules from one window.

Final Application Screenshot

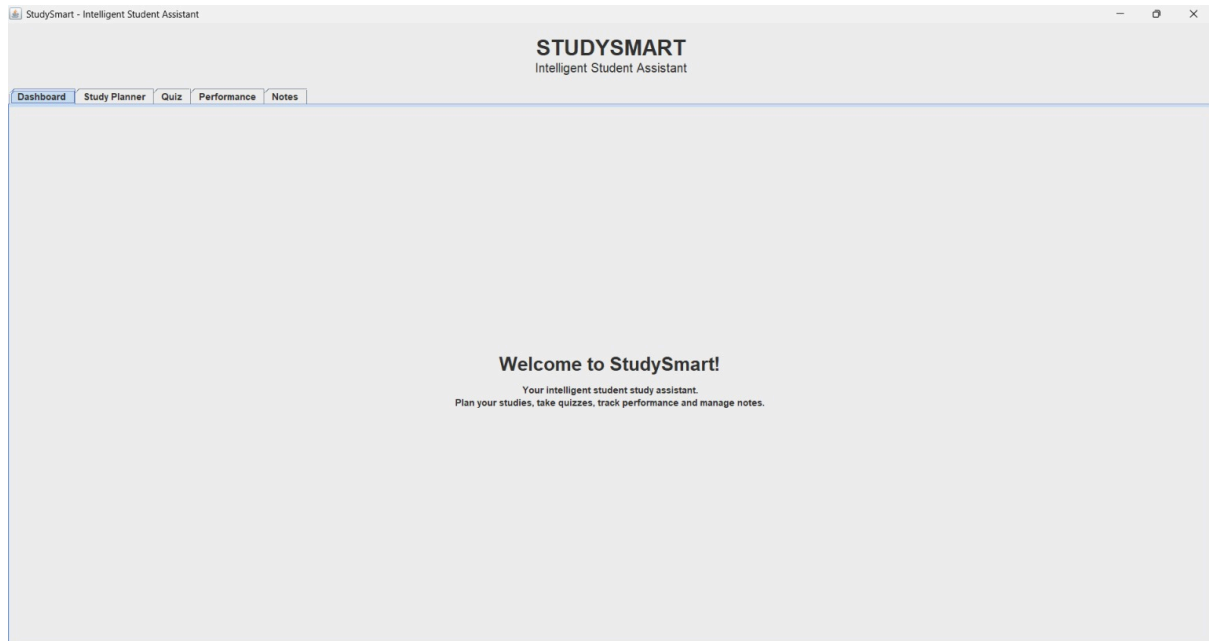


Figure 10: Final StudySmart Application

23. CHALLENGES FACED DURING DEVELOPMENT

While developing this project, I faced a few practical problems.

- 23.1 Managing Multiple Java Files : Initially, keeping track of different classes and packages was confusing. Separating the project into **model**, **service**, and **gui** packages made the structure easier to understand.
- 23.2 Connecting GUI With Logic : Creating the GUI was not enough. The buttons also had to perform the required operations. For example, clicking the Add Study Task button had to create a **StudyTask** object and add it to the planner.
- 23.3 Handling User Input : The program had to handle cases such as empty fields and invalid duration values.
- 23.4 File Handling : Saving and reading notes required understanding Java file handling and exception handling.
- 23.5 Debugging : During development, some errors were related to package names, class paths, and compilation. Testing each part separately made it easier to find and fix these problems.

24. LEARNING OUTCOMES

This project gave me practical experience with several Java concepts.

Some of the main things I learned are:

- How to divide a project into multiple classes.
- How packages can be used to organize Java code.
- How constructors, getters, and setters work.
- How encapsulation is implemented.
- How to use `ArrayList`.
- How objects can be stored and processed using collections.
- How to handle invalid input.
- How to use `try-catch` for exception handling.
- How to read and write files in Java.
- How Java Swing components work.
- How button events can be handled.
- How different classes can be connected together.
- How to test individual modules before integrating them.

Most importantly, I got a better understanding of how the concepts from a Java course come together while building a complete application.

25. FUTURE ENHANCEMENTS

There are several features that can be added to StudySmart in the future.

Some possible improvements are:

1. Adding a database instead of using a text file for storage.
2. Adding user login and registration.
3. Allowing users to create their own quiz questions.
4. Adding more subjects and question categories.
5. Adding graphical charts for performance.
6. Adding reminders for study tasks.
7. Adding a proper calendar-based study planner.

8. Improving the recommendation system with more rules.
 9. Adding the option to export notes and performance reports.
 10. Creating a web or mobile version of the application.
-

26. CONCLUSION

StudySmart was developed as a practical Java project to bring different study-related activities together in one application.

The project includes a Study Planner, Quiz System, Performance Analyzer, Recommendation Engine, and Notes Manager. These modules are connected through a Java Swing graphical interface.

While developing the project, I was able to apply concepts such as Object-Oriented Programming, collections, methods, packages, file handling, exception handling, and GUI programming.

The project also helped me understand that developing an application is not only about writing individual programs. The different classes and modules have to work together properly, and testing and debugging are equally important.

StudySmart can be further improved by adding database storage, authentication, more advanced analytics, reminders, and additional study features.

27. REFERENCES

1. Java Development Kit (JDK) documentation.
 2. Java Swing documentation.
 3. Visual Studio Code documentation.
 4. GitHub documentation.
 5. Programming in Java course concepts and classroom material.
-

THANK YOU.....

